

INF1103: Programming Fundamentals with DevOps

Week 8: Introduction to Object Oriented Programming

About me

- Dr Oran Zane Devilly, PhD

  Associate Professor, SIT

  Deputy Director / Research, Lead STLA



- Roles for this module:
  - Module Lead INF1103
  - Lecture Delivery (OOP part of module)
  - Lab Prep (OOP part of module)
  - Project Prep
  - Spend time in all the many lab groups

- If any module related matter, please email me at:

  Oran.Devilly@singaporetech.edu.sg

About me

- Research Interests
  - Artificial Intelligence
  - Educational Research / Educational Technology
  - VR / AR
  - Gamification
  - Game Based Learning

- Hobbies
  - Travel
  - Gaming
  - Space the universe and everything
  - Rugby





Do not edit
How to change the design



How good do you feel you are at programming to date

ⓘ The Slido app must be installed on every computer you're presenting from

slido

Learning Outcomes

- Object Oriented Programming Foundations
- Classes and Objects
- Encapsulation
- Class Diagram
- S.O.L.I.D

BUT FIRST, a concept

- A **variable (A box!)** stores **information(data)**, each variable stores 1 specific **type** of information

- Python doesn't ask for the type but it decides it based on what you put in the **variable (the box!),** this is called **dynamic typing**

- Many languages expect the programmer to write the type:
  - int num1 = 10
  - char letter = 'R'
  - string name = 'Bob'

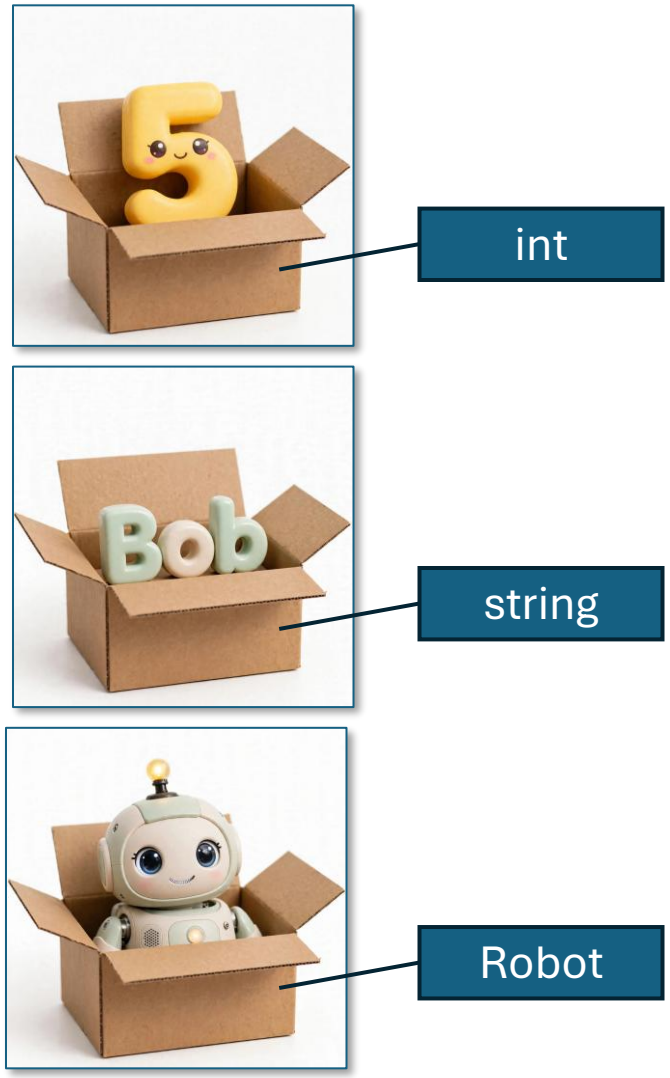
- Example, all of the following num variables are different types!
  - num = 5 # Integer (int)
  - num = "five" # String (str)
  - num = 5.0 # Float (float)

BUT FIRST, a concept

- Int, string, float, double, these are all consider simple data types and store single values
- More complex data types include:
  - Collection types: list, tuple, set, dict
  - Custom types: classes and objects
- You have already seen lists and dictionary types
- Today we will start to explore classes and object types
- SO… does a robot fit in a box?

BUT FIRST, a concept

- So, how do we put an object into a box?
  - num = 5
  - name = “Bob”
  - robot1 = Robot()



Yes, a robot fits in a box
In fact, any object does!

Good news!

- AI isn’t great at software system design
- AI currently needs a lot of guidance
- Managers are in-demand, junior developers are not
  - So, how to become a manager?
  - Get GOOD!
- Human first approach
  - Cognitive offload is bad

Object Oriented Programming Foundations

- **Procedural Programming:**
  - Code is organised around process/actions that occur in a program
  - Procedural programming structures a program around functions and procedures
  - With procedural programming you could have one file with everything in it, or split functions between files
  - Common to have a main file that talks to functions and passes data around

Object Oriented Programming Foundations

- **Procedural Programming:**
  - If data structures change, many functions must also be changed
  - Programs are based on complex function hierarchies which make them:
    - Difficult to understand and maintain
    - Difficult to modify and extend
    - Easy to break

- Good programming practices, code should be:
  - Readable, maintainable, reusable, reliable, robust, testable, scalable, efficient, modular, flexible, simple, understandable, extensible (to name but a few)

Object Oriented Programming Foundations

- Object Oriented Programming:
  - OOP is based around objects that combine data (state) with the operations (methods) that operate on that data.
  - Code is organised around things/entities that have both data and behaviour
  - Objects are instances of classes that define the data and operations associated with them.
  - Even though we can maintain good programming practices in procedural programming, OOP makes them much easier.

Object Oriented Programming Foundations

- Each object represents whatever exists inside your program
  - Images, cars, animals, banks, players, people, fruits, tables

- Most contemporary languages support object-oriented programming approaches
  - Python, Java, C++, C#, VB, etc.

Object Oriented Programming Foundations

Code Quality	Procedural	OOP
Maintainable	△	✓
Reusable	△	✓
Reliable	△	✓
Robust	△	✓
Testable	△	✓
Scalable	△	✓
Efficient	✓	✓
Modular	△	✓
Flexible	△	✓
Simple	✓	△
Extensible	△	✓

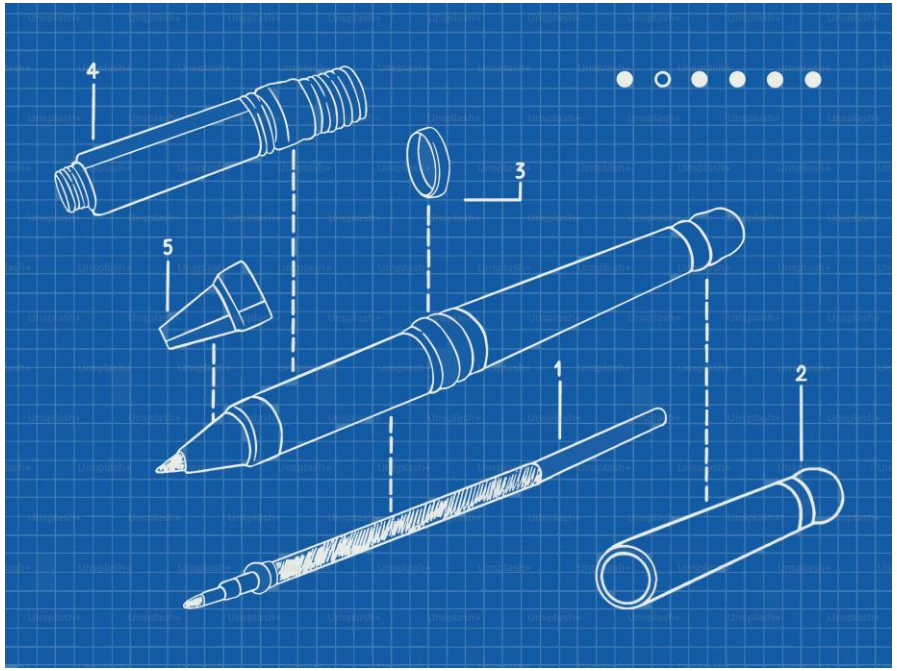
Object Oriented Programming Foundations

- Ok sounds great, but:

- ***WHAT IS OBJECT ORIENTED PROGRAMMING!***

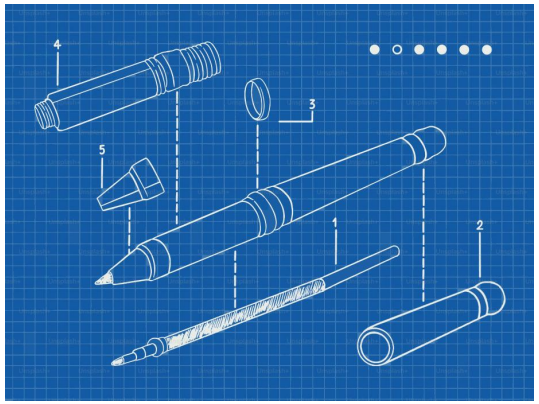
Classes and Objects

- Lets start simple:
  - In the beginning the was… a class
  - A class is like a blueprint, it defines what an object is and what it can do, but it isn't yet an object



- Class:
  - It defines what an object is but is not an object
  - It has a list of behaviours (methods which are basically functions)
  - It has its own attributes (variables) about the object
  - Every object we create from this blueprint will have the same behaviours and attributes

Classes and Objects



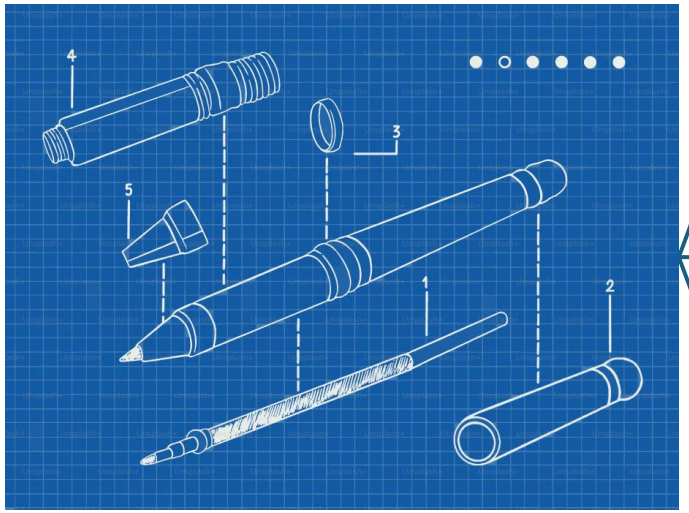
Pen
- penColour : String - inkColour : String
+ write() : void

- Class Pen (super simple):
  - Behaviours: It can write in a certain color
  - Attributes:
    - The case colour
    - The ink colour

- This is a class diagram:
  - Part of a set of diagrams from UML (Unified Modelling Language)
  - Represents full class structure
  - Shows the class name (top box) attributes (middle box) and methods (bottom box)
  - Minus '-' means private and Plus '+' means public (more on this later)
  - Must always have 3 boxes! (Even if no attributes or no methods)
  - Should make sense against the code you write (should help you figure out what code to write)

Classes and Objects

- Once we are happy with our class, we can now instantiate it
  - Create the object from the class (1 class can create as many objects as you want)





pen1



pen2



pen2

Each pen has:

- Write method
- Case Colour Attribute
- Ink Colour Attribute

Each pen can:

- Have a different case colour
- Write in a different ink colour
- All pens can write

Classes and Objects

Pen
- penColour : String - inkColour : String
+ write()

- So what does that look like in code?

```
class Pen:
        def __init__(self, penColour, inkColour)
            self.penColour = penColor
            self.inkColor = inkColour

        def write(self):
            print(f"I am a {self.penColor} pen and I am writing in {self.inkColour}")

pen1 = pen("silver", "black")
pen2 = pen("black", "blue")
pen3 = pen("gold", "red")
pen1.write()
pen2.write()
pen3.write()
```

'class' defines a class with a name

Init is initializer (constructor) takes parameters from outside the class.

self.penColour is an attribute, penColour(no 'self') is a local variable

Functions inside classes are called methods!

This code is not in the class, but uses the class to create pen objects

Classes and Objects

- In object oriented languages the entry point to the code when and object is created (instantiated) is called a constructor
- In python it is called the initializer, always has a ‘self’ parameter and usually has other parameters set by external code
  - def __init__ (self, penColour, inkColour)
- self refers to the object itself, so if we were talking to pen1, self would be a reference to pen1, if we were talking to pen2, self would be a reference to pen2
- External code means, code not from this class, it could be our main function, it could be another class, etc.

Encapsulation

- Encapsulation is basically about hiding internal implementation of a class from external code (data hiding)
- Encapsulation uses 3 main access specifiers which decides ‘who’ can see the data:
  - Public : Can be seen / used by external code
  - Private : Can only be seen / used by the class which declared it
  - Protected : Can be seen/used by class which declared it and child classes (generally, slightly different behaviors per language)

- In UML public is + private is – and protected is #

Encapsulation

- In Python, Encapsulation is more conceptual than enforced, as a developer you should still think about what external users can and can't access in your class
- self defines a variable as accessible by the whole class, usually declared in __init__
  - self.myvar is public
  - self.__myvar is intended for private internal class use only
  - self._myvar is intented for internal / protected-style use
- Again, remember, python doesn't enforce these rules, however other languages (java, C++, C#) do.

Encapsulation

- So why do we need to think about encapsulation
  - Helps us consider data and how it is / should be manipulated
  - Controlling how states of objects are accessed and modified
  - Considering what parts of an object others should and shouldn’t interact with
  - What others need to know and don’t need to know about your class (not all implementation details are needed by all people!)
  - Helps with code organization (arrange code visually before implementing)
  - Extensibility (can update parts that others don’t see as long as the parts they do access don’t change)
  - Supports code flexibility, maintainability and extensibility

Class Diagram (UML)

Pen
- penColour : String - inkColour : String
+ write() : void

- Class Diagram Check List
  - It is 3 boxes in order for
  1. Class Name
  2. Attributes (AKA Member Variables)
  3. Methods
- ‘-’Means private, to be accessed only inside the class
- ‘+’ means public, can be accessed by external code
- ‘#’ means protected, can be accessed by external code

- : String refers to data types, in our example both are String (text), or void
- Parameters for methods should also be included
- Return types in methods should also be included, in this case write() returns nothing so we write void

Note: Python doesn’t require explicit data type declaration and doesn’t enforce access specifiers, but UML should be ‘universal’ information for any language

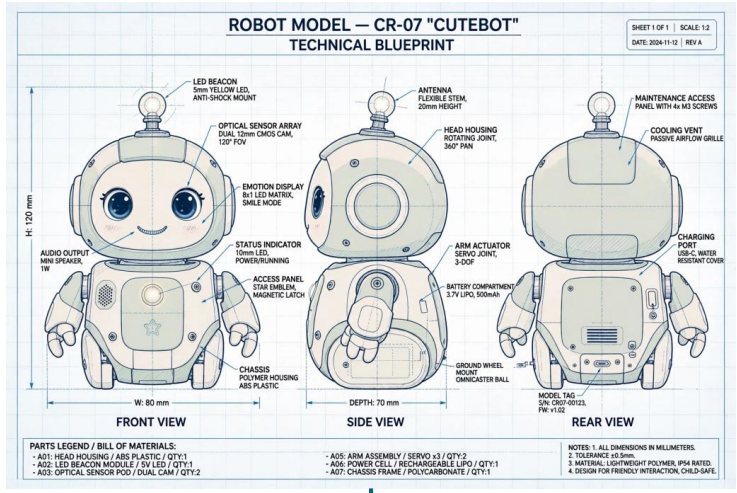
Conceptual Example [The Robot]

- How might we build a robot?
  - What is a robot?
  - What information does a robot need to keep track of?
  - What can the robot do?
  - Who talks to the robot?
  - What should those that talk to the robot be able to access?

Conceptual Example [The Robot]

What is a robot?

- A robot in our case is an object
- To create an object we need a class
- A class allows us to create as many unique robots as we want
- Before we create the objects though, lets work on that blueprint!

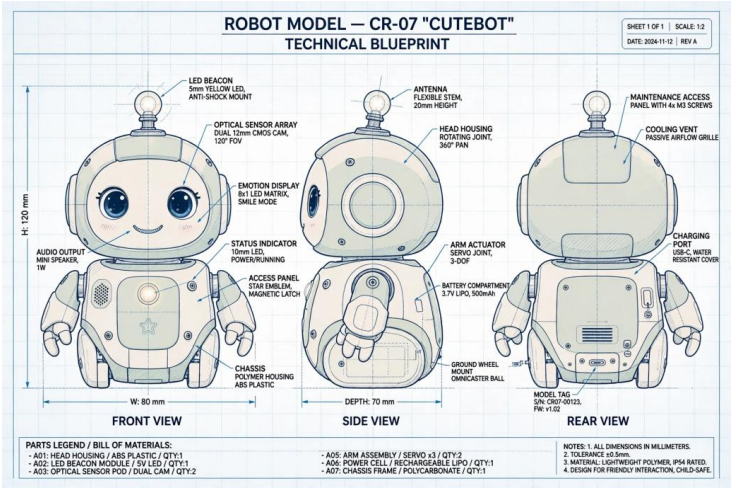






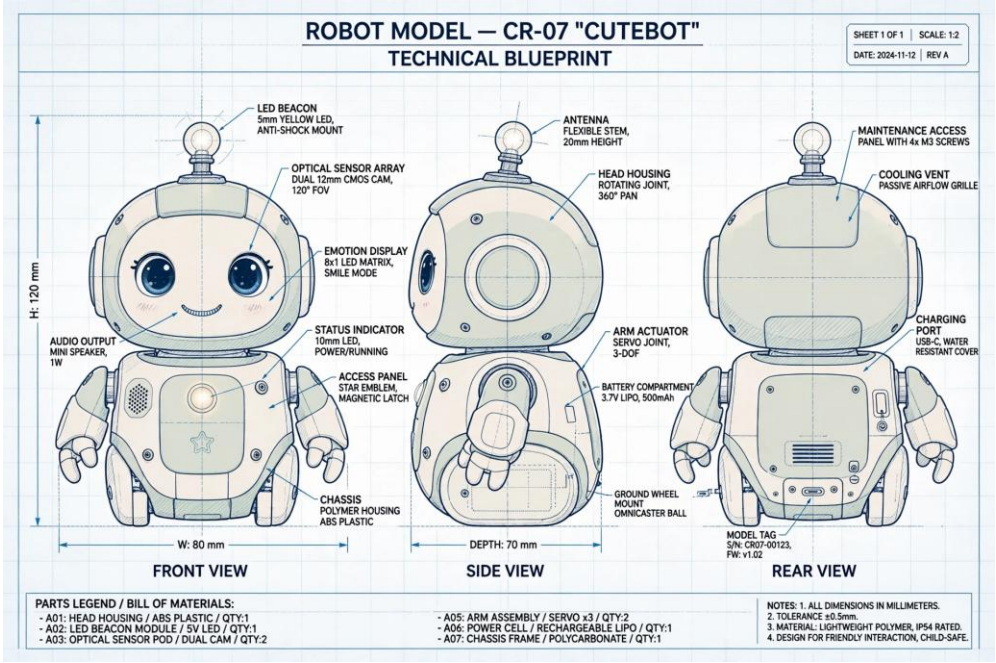
Conceptual Example [The Robot]

- What information does a robot need to keep track of?
  - Name
  - BatteryLevel
  - X coordinate
  - Y coordinate
  - isOn (perhaps Status is better?)



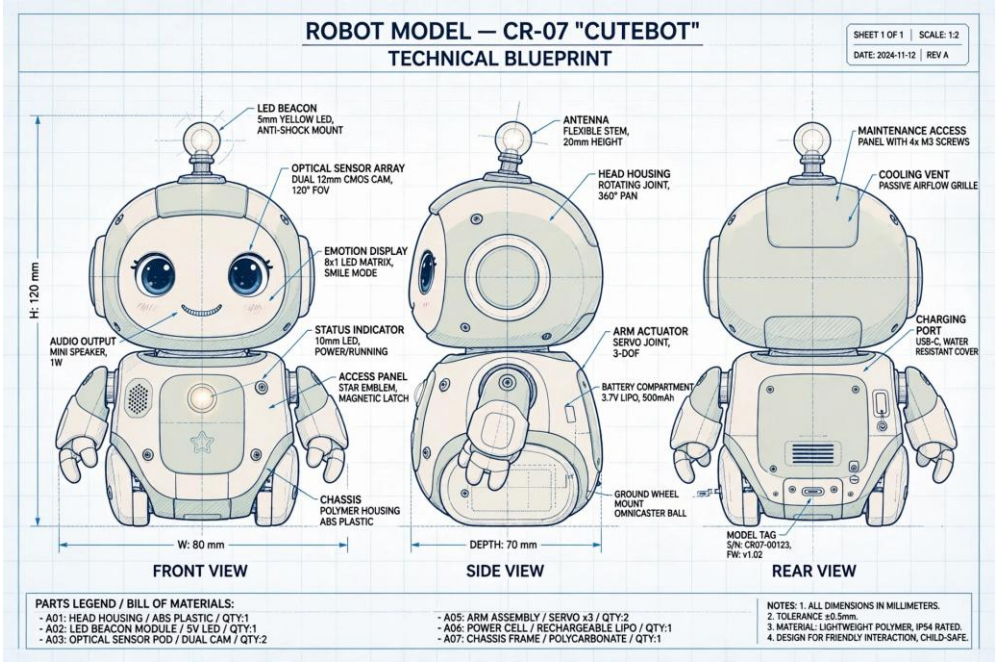
Conceptual Example [The Robot]

- What can the robot do?
  - turnOn()
  - turnOff()
  - Move()
  - Charge()
  - Pickup()
  - Drop()
  - getStatus()



Conceptual Example [The Robot]

- Who talks to the Robot?
  - Who as two thumbs and talks to a robot



Conceptual Example [The Robot]

- Who talks to the Robot?
  - Who as two thumbs and talks to a robot

Conceptual Example [The Robot]

- Who talks to the Robot?
  - Who has two thumbs and talks to a robot
  - But maybe I'm not talking directly to a robot?
  - We could have a controller that knows what the robot can do, but doesn't need to know how to do it



Conceptual Example [The Robot]

- Who talks to the Robot?
  - A robot controller
  - It knows what the robot can do
    - turnOn, move, pickup, getStatus
  - It can request the robot (any robot) to do those actions.
  - I talk to the controller, not the robot

Conceptual Example [The Robot]

- What should those that talk to the Robot be able to access
  - ME:
    - Just commands or maybe not even exact commands (LLM anyone?)
  - Robot_Controller
    - Can call the ‘public’ functions of the robot, the end points that we should be able to talk to
    - Example should change X, Y directly, just give a target coordinate
  - Robot
    - Deals directly with implementation, has functions it uses for this
    - Could have an algorithm for how to change its X,Y directly
    - Decides how it will do what the controller asks, but doesn’t have to tell the controller how it does it
    - Can give feedback (success, failure, standing by)

Conceptual Example [The Robot]

Robot
- name : String - batteryLevel : int - x : int - n : int - status : String
+ turnOn() : void + turnOff() : void + move (x:int, y:int):void + charge() : void + getStatus() : String

What you see in class design is What you get in the code!

```python
class Robot:

    def __init__(self, name):
        self._name = name
        self._batteryLevel = 100
        self._x = 0
        self._y = 0
        self._status = "OFF"

    def turnOn(self):
        self._status = "IDLE"
        print(f"{self._name} is now ON.")

    def turnOff(self):
        self._status = "OFF"
        print(f"{self._name} is now OFF.")

    def move(self, x, y):
        if self._status == "OFF":
            print(f"{self._name} is OFF. Turn it on first.")
            return

        if self._batteryLevel <= 0:
            print(f"{self._name} has no battery.")
            return

        self._status = "MOVING"
        self._x = x
        self._y = y
        self._batteryLevel -= 10
        self._status = "IDLE"

        print(f"{self._name} moved to ({self._x}, {self._y}).")

    def charge(self):
        self._status = "CHARGING"
        self._batteryLevel = 100
        self._status = "IDLE"

        print(f"{self._name} is fully charged.")

    def getStatus(self):
        return (
            f"Robot: {self._name}\n"
            f"Position: ({self._x}, {self._y})\n"
            f"Battery: {self._batteryLevel}%\n"
            f"Status: {self._status}"
        )


# External code
robot = Robot("R2D2")

robot.turnOn()
robot.move(10, 5)

print(robot.getStatus())

robot.move(20, 15)

print(robot.getStatus())

robot.charge()

print(robot.getStatus())

robot.turnOff()
```

S.O.L.I.D

- So, how to design good classes
- Would you like to be SOLID at object oriented programming?
- There are some approaches to Object Oriented that are tried and tested that developers have documented over time to support those who want to be SOLID at programming

S.O.L.I.D

- **S**ingle Responsibility: A class should have one clear responsibility.
- **O**pen/Closed: Open for extension, closed for modification.
- **L**iskov Substitution: Subclasses should be usable wherever their parent class is expected.
- **I**nterface Segregation: Don't force classes to depend on methods they don't need.
- **D**ependency Inversion: Depend on abstractions, not concrete implementations.

S.O.L.I.D

- The letter for today is 

- When building your class, ask yourself what is the single objective that this class is designed to achieve

- As the class grows, ask yourself, does this class still meet the single objective
  - Yes, you are still satisfying the single responsibility principle
  - No, you have violated the principle!!!!

S.O.L.I.D

- **Violated the principle**
  - It is a pretty strong word, but remember they are principles and not rules
  - Try to follow principles, if your code starts to become very convoluted then let the principle go
  - For Single Responsibility Principle SRP however, you should be able to follow it quite consistently

What's next

- Lab on Friday (project presentations, basic classes / objects lab)
- Week 9 – OO Relationships Part 1 (Inheritance / Abstract Classes / Interfaces)
- Week 10 – Association / Aggregation / Composition / Polymorphism
- Week 11 – Software Design Patterns / Error Handling / CI|CD pipeline
- Week 12 – No lecture (Wednesday submission / Friday Demo)
- Week 13 – Quiz (On campus)

Project Overview (for humans)

- Brief will be released after class, but highlights first:
  - Refactor Phase 1 into well designed object oriented application
  - Demonstrate all OOP concepts from week 8-11
  - Preserver the four mandatory managers responsible for the AI-centered data pipeline
  - Add one useful new features that extend the original application
  - Use automated tests (Git, Docker, CI)
  - Select one additional feature that goes beyond the lecture notes

- Follow SOLID principles (SOL is the focus for this module)

Project Overview

- Submission deliverables similar to part 1:
  - OOP source code
  - Engineering Report (5 pages)
  - Automated Tests
  - Git Repository URL showing meaningful incremental history and CI workflow
  - Docker Setup
  - New Feature Notes

- Presentation face to face, Friday week 12

Thanks!!!

